EE2002-Tutorial 4 Solutions

1. Consider a logic circuit with three inputs, namely $x, y,$ and z, and three outputs, namely $A, B,$ and C. When the binary input (i.e., xyz) is 0, 1, 2, or 3, the binary output (ABC) should be one greater than the input. For example, if $xyz = 000$, then $ABC = 001$. When the binary input is 4,5,6, or 7, the binary output should be one less than the input. Design a circuit to realize this using only NAND gates.

Solution:

The design procedure consists of a few steps.

Step 1: Derive the truth table that defines the required relationship between inputs and outputs.

Inputs (x, y, z)	**Outputs** (A, B, C)
000	001
001	010
010	011
011	100
100	011
101	100
110	101
111	110

Step 2:The Boolean expressions for outputs A, B, and C are:

$$A = \sum(3, 5, 6, 7)$$

$$B = \sum(1, 2, 4, 7)$$

$$C = \sum(0, 2, 4, 6)$$

Step 3: Obtain the simplified expressions for the each output as function of the input variables.

x \ yz	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Result

$F = xy + xz + yz$

Figure 1: K-map for A

x \ yz	00	01	11	10
0	0	1	0	1
1	1	0	1	0

Result

$F = xyz + x\bar{y}\bar{z} + \bar{x}y\bar{z} + \bar{x}\bar{y}z$

Figure 2: K-map for B

x \ yz	00	01	11	10
0	1	0	0	1
1	1	0	0	1

Result

$F = \bar{z}$

Figure 3: K-map for C

Step 4: Realize the circuit using NAND gates

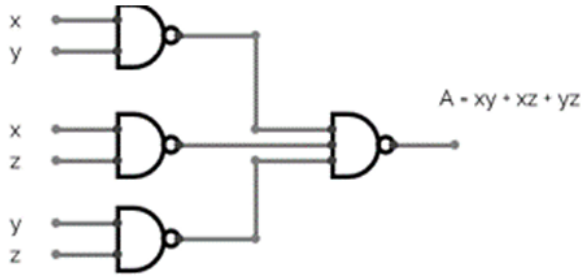


Figure 4: NAND implementation for A

$B = x'y'z + x'yz' + xy'z' + xyz$

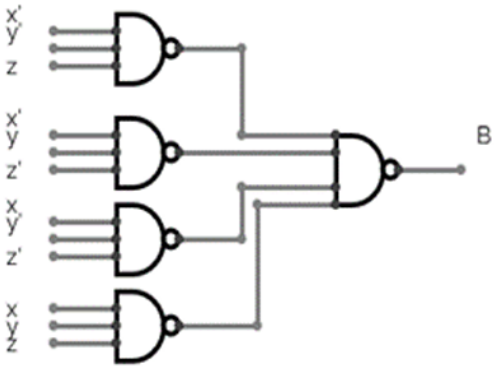


Figure 5: NAND implementation for B

$C = z'$

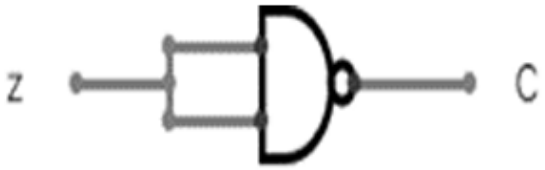


Figure 6: NAND implementation for C

Note: In the circuit for realization of B, the complements at the input can also be realized readily using NAND gates.

2.Design a 2-input OR gate using the minimum number of Half Adders (HAs).

Solution:

A Half Adder (HA) has the following Sum and Carry equations:

$$\text{Sum}(S) = A \oplus B = AB' + A'B$$
$$\text{Carry}(C_{out}) = AB$$

To obtain an OR gate using the minimum number of Half Adders, we proceed as follows:

Step 1: First Half Adder The first HA takes A and B as inputs and generates:

$$S_1 = A \oplus B = AB' + A'B$$
$$C_1 = AB$$

Step 2: Second Half Adder The Sum (S_1) and Carry-out (C_1) of the first HA are given as inputs to a second HA:

$$S_2 = S_1 \oplus C_1$$

Expanding this using the expressions for S_1 and C_1:

$$S_2 = (AB' + A'B) \oplus AB$$

Using the XOR property:

$$\begin{aligned} S_2 &= (AB + A'B')AB + (A' + B')(A'B + AB') \\ &= AB + A'B + AB' \\ &= A + B \end{aligned}$$

Thus, the output S_2 of the second HA is $A + B$, which is the OR function.

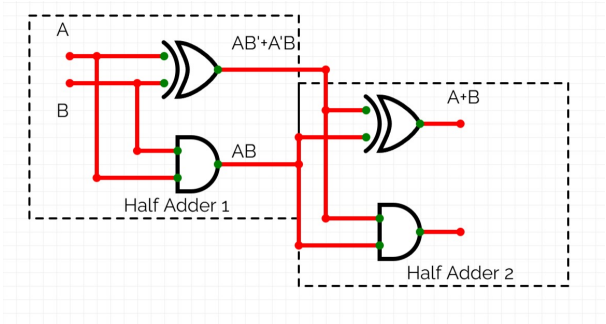


Figure 7: Implementation of a 2-input OR gate using two Half Adders

3.For the 4-bit Ripple Carry Adder, compute the sum and carry at each stage for the given binary inputs:

$$a_3a_2a_1a_0 = 1011_2, \quad b_3b_2b_1b_0 = 1101_2$$

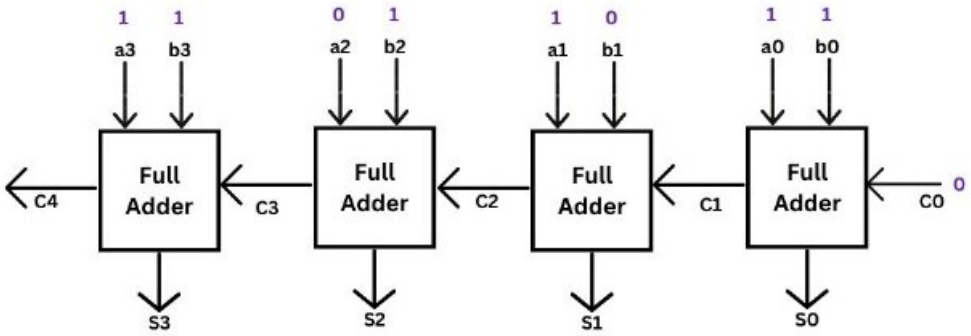


Figure 8: 4-bit Ripple Carry Adder

Solution:

A Full Adder (FA) is described by the following Sum and Carry equations:

$$\text{Sum}(S) = a \oplus b \oplus c_{in}$$
$$\text{Carry}(c_{out}) = ab + bc_{in} + ca = (a \cdot b) + (c_{in} \cdot (a \oplus b))$$

Each full adder follows the equations:

$$s_i = a_i \oplus b_i \oplus c_i$$

$$c_{i+1} = (a_i \cdot b_i) + (c_i \cdot (a_i \oplus b_i))$$

Assume the initial carry $c_0 = 0$.

Bit Position	a_i	b_i	c_i (Carry-in)	Sum s_i	Carry-out c_{i+1}
0 (LSB)	1	1	0	$1 \oplus 1 \oplus 0 = 0$	$(1 \cdot 1) + (0 \cdot (1 \oplus 1)) = 1$
1	1	0	1	$1 \oplus 0 \oplus 1 = 0$	$(1 \cdot 0) + (1 \cdot (1 \oplus 0)) = 1$
2	0	1	1	$0 \oplus 1 \oplus 1 = 0$	$(0 \cdot 1) + (1 \cdot (0 \oplus 1)) = 1$
3 (MSB)	1	1	1	$1 \oplus 1 \oplus 1 = 1$	$(1 \cdot 1) + (1 \cdot (1 \oplus 1)) = 1$

Table 1: Bitwise Computation of the Ripple Carry Adder

Figure 9 shows the inputs and outputs for 4-bit ripple carry adder

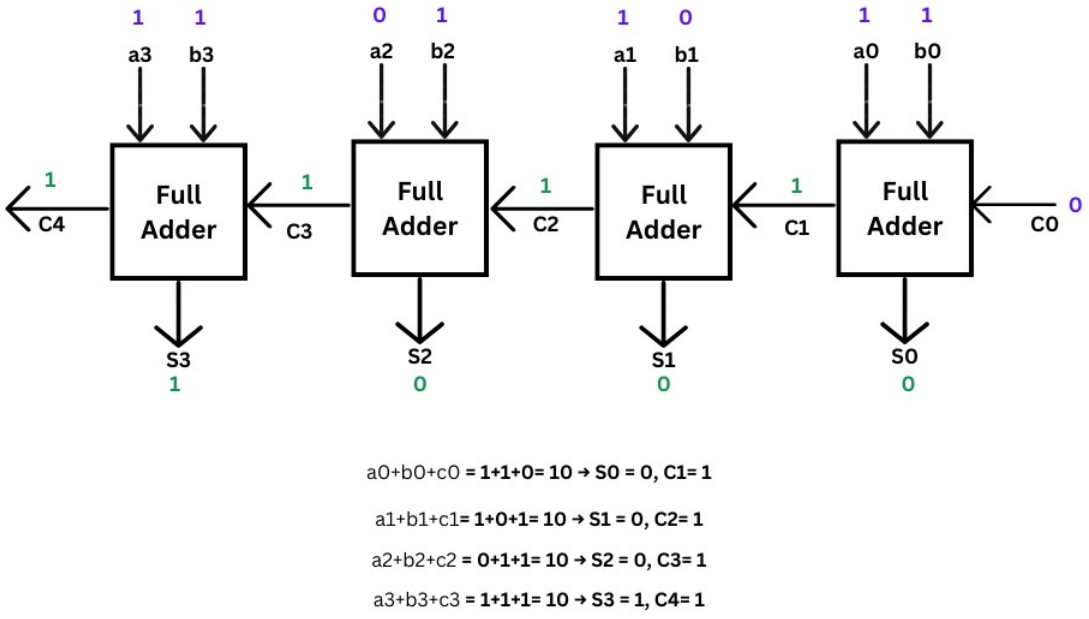


Figure 9: 4-bit Ripple Carry Adder Computation

4.Consider the same binary inputs as in question 3. Assume that we have a 4-bit carry lookahead adder (instead of a ripple carry adder). Perform the addition showing the numerical values of the generate and propagate signals as well as the carry calculations using the generate and propagate equations.

Solution:

4-bit Carry Lookahead Adder Computation

$$A = a_3a_2a_1a_0 = 1011_2$$
$$B = b_3b_2b_1b_0 = 1101_2$$

Assume initial carry: $C_0 = 0$.
The generate G_i and propagate P_i signals are computed as:

$$G_i = a_i b_i$$
$$P_i = a_i \oplus b_i$$

Bit Position i	a_i	b_i	$G_i = a_i b_i$	$P_i = a_i \oplus b_i$
0	1	1	1	0
1	1	0	0	1
2	0	1	0	1
3	1	1	1	0

Table 2: Generate and Propagate Signal Values

Carry Computations

Using the carry equations:

$$C_1 = C_0P_0 + G_0$$
$$C_2 = (C_0P_0 + G_0)P_1 + G_1$$
$$C_3 = [(C_0P_0 + G_0)P_1 + G_1]P_2 + G_2$$
$$C_4 = C_0P_0P_1P_2P_3 + P_3P_2P_1G_0 + P_3P_2G_1 + P_3G_2 + G_3$$

Substituting values:

$$C_1 = (0 \cdot 0) + 1 = 1$$
$$C_2 = (1 \cdot 1) + 0 = 1$$
$$C_3 = (1 \cdot 1) + 0 = 1$$
$$C_4 = (1 \cdot 0) + 1 = 1$$

Sum Computation

The sum bits are computed as:

$$S_i = P_i \oplus C_i$$

Bit Position i	P_i	C_i	$S_i = P_i \oplus C_i$
0	0	0	0
1	1	1	0
2	1	1	0
3	0	1	1

Table 3: Sum Bit Values

5.Design a subtractor unit using a 4-bit comparator, 4-bit adder and some additional gates, which performs $A-B$ if $A > B$ and $B-A$ otherwise. Assume A and B are two 4-bit numbers.

Solution:

In this design, we need to implement a 4-bit subtractor unit that performs:

$$\text{Result} = \begin{cases} A - B, & \text{if } A > B \\ B - A, & \text{otherwise} \end{cases}$$

where A and B are two 4-bit binary numbers.
Subtraction of two binary numbers can be done using 2's complement:

$$A - B = A + (-B)$$

That is, to subtract B from A, we compute the 2's complement of B and add it to A. The 2's complement of a number B is given by:

$$-B = \overline{B} + 1$$

where $\overline{B}$ is the bitwise NOT (1's complement) of B. If $A \leq B$, then we compute $-A = \overline{A} + 1$.
The subtractor unit consists of:

- A 4-bit comparator to determine the relationship between A and B.
- A 4-bit binary adder to perform the addition operation.
- External logic gates (XOR, OR) to control the input selection dynamically as showin below figure.

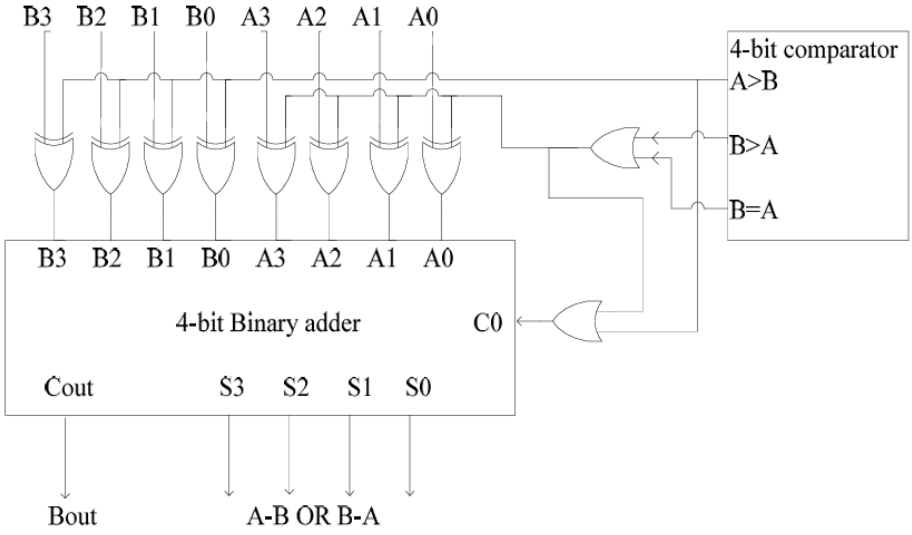


Figure 10: 4-bit Subtractor Unit

We have a few cases as follows.
Case 1: When $A > B$

- The output of the 4-bit comparator indicates $A > B$.
- The 1's complement of B is taken from the XOR gate.
- The comparator output ($A > B$) is connected to the carry-in (C_0) of the 4-bit adder.
- The result is computed as $A + (-B) = A - B$.

Case 2: When $A \leq B$

- The output of the comparator for $A < B$ or $A = B$ is passed through an OR gate, ensuring a (logic) high signal in both cases.
- The 1's complement of A is taken from XOR gate .
- The OR gate output is connected to the carry-in (C_0) of the 4-bit adder.
- The result is computed as $B + (-A) = B - A$.

6.Design an adder/subtractor unit using a 4-bit binary adder and some external gates, which gives out $A + B$ if $C = 0$ and $A - B$ if $C = 1$. Also, provide an indicator for checking the overflow.

Solution:

Design a 4-bit adder/subtractor unit that:

- Performs $A + B$ if $C = 0$.
- Performs $A - B$ if $C = 1$.
- Provides an overflow indicator for signed operations.

Figure 11 shows the circuit for the 4-bit Adder/Subtractor:

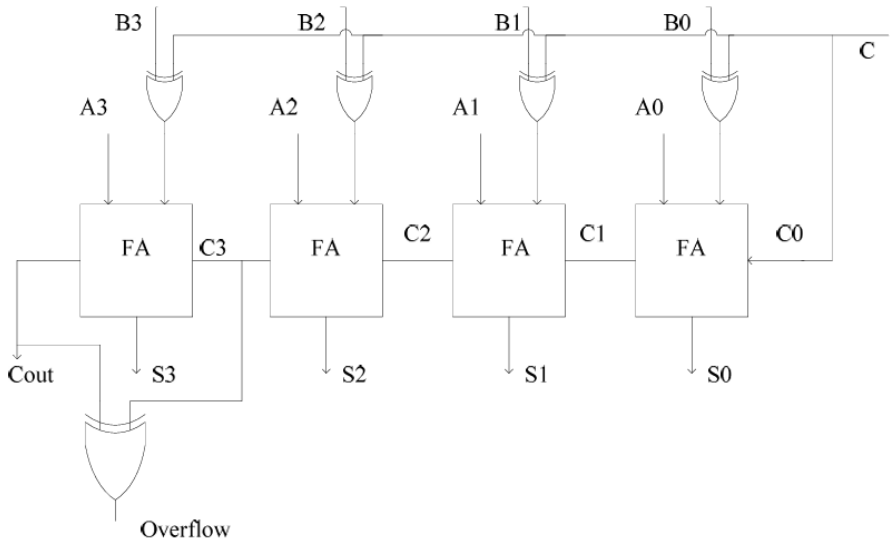


Figure 11: 4-bit Adder/Subtractor Circuit

The circuit consists of:

- A 4-bit binary adder.
- XOR gates to invert B when $C = 1$.
- An overflow detection circuit.

To perform subtraction using a binary adder, we use the two's complement method:

$$A - B = A + (-B)$$

where:

$$-B = \overline{B} + 1$$

Each bit of B is XORed with C to determine whether it should be inverted:

$$X_i = B_i \oplus C$$

The 4-bit adder then computes:

$$\text{Sum} = A + X + C$$

where C acts as the carry-in.
Overflow Detection: For signed numbers, overflow is handled as:

$$\text{Overflow} = C_{\text{out}} \oplus C_{\text{N-1}}$$

where:

- $C_{\text{N-1}}$ is the carry from the second-last bit.
- Overflow occurs if adding two positive numbers gives a negative result.
- Overflow also occurs if adding two negative numbers gives a positive result.

7. You are given a 2:4 decoder, a 3-input OR gate and an inverter. Using these components, design a system that takes A and B as inputs and generates the following four outputs:

$$AB, \quad (AB)', \quad A + B, \quad (A + B)'$$

Solution:

Step 1: Some of the outputs can be obtained as follows.
A 2:4 decoder has two inputs (A, B) and four outputs, which correspond to the minterms of the inputs:

$$\begin{aligned} D_1 &= AB \\ D_2 &= AB' \\ D_3 &= A'B \\ D_4 &= A'B' \end{aligned}$$

From this, we directly obtain:

$$\begin{aligned} AB &= D_1 \\ (A + B)' &= A'B' = D_4 \end{aligned}$$

Step 3: Generating $(AB)'$
We use De Morgan's theorem:

$$\begin{aligned} (AB)' &= A' + B' \\ &= A'(B + B') + B'(A + A') \\ &= A'B + A'B' + AB' + A'B' \\ &= A'B + AB' + A'B' \\ &= D_3 + D_2 + D_4 \end{aligned}$$

The terms $A'B$ (D_3), AB' (D_2), and $A'B'$ (D_4) are all available from the decoder outputs. Thus, we use the 3-input OR gate:

$$(AB)' = D_3 + D_2 + D_4$$

The complete circuit is shown in Figure 12.

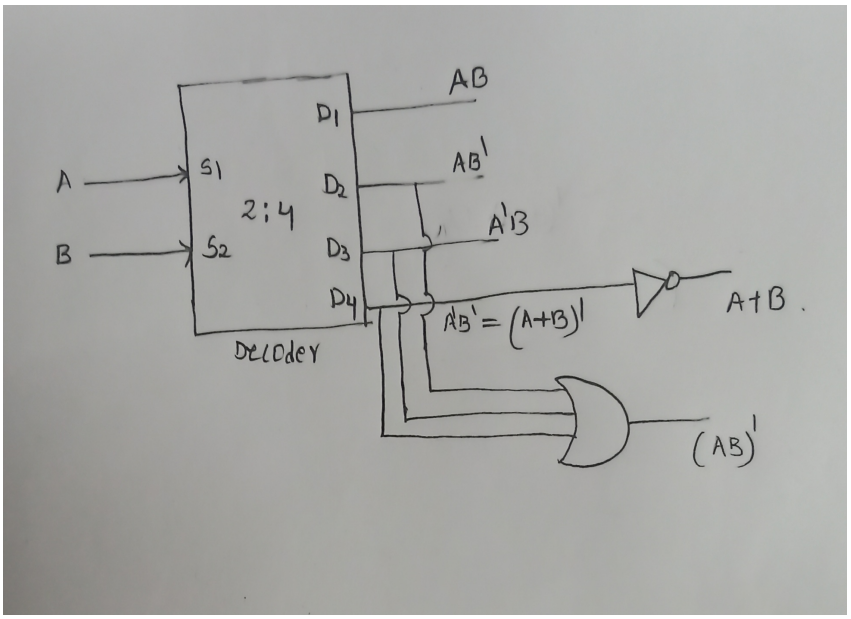


Figure 12: Implementation of function using 2:4 decoder

8. Implement the function $f(A, B, C) = \sum m(3, 5, 6, 7)$ using a 3-to-8 decoder and any additional logic gates (as required).

Solution:

The given function is:

$$f(A, B, C) = \sum m(3, 5, 6, 7)$$

This means that the function is high (1) for minterms 3, 5, 6, and 7.
A 3-to-8 decoder has three inputs (A, B, C) and eight outputs, where each output corresponds to a specific minterm. The solution requires activating the appropriate minterms and combining them via an OR gate. Figure 13 depicts the solution.

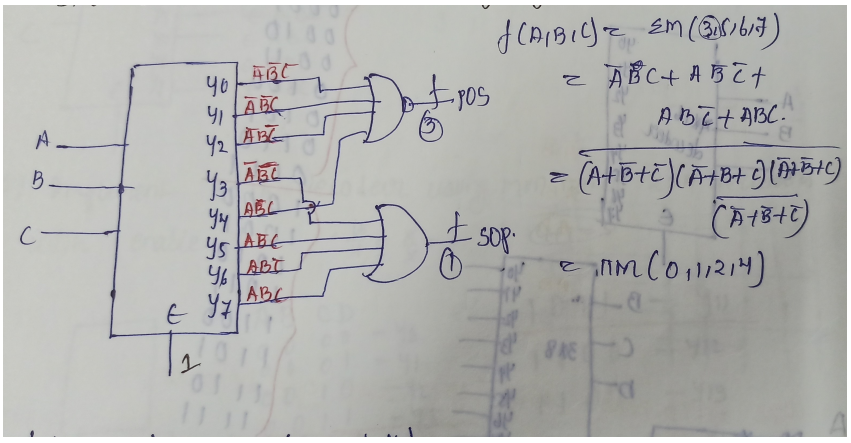


Figure 13: Implementation of function using 3-to-8 decoder